

# Demystifying Java language features from 8 and 9

**Ionuț Baloșin**

Software Architect  
@ionutbalosin



# About Me

## Ionuț Baloșin

**Software Architect** @ luxoft.com

**Technical Trainer**

- “Java Performance and Tuning” course
- “Software Architecture” course

ionutbalosin.com



@ionutbalosin

To demystify - to make something easier to understand

[dictionary.cambridge.org](https://dictionary.cambridge.org)

To demystify - to make something easier to understand

[dictionary.cambridge.org](https://dictionary.cambridge.org)

**My definition:** try to make Java language features easier to understand by digging into the JVM internals

***“Explore the world. Nearly everything is really interesting if you go into it deeply enough.”***



**Richard Feynman**

# Agenda

Lambdas

Function, Predicate, Supplier, Optional

Method references

Streams

String in Java 9

StackWalker

Collection factory methods

Contended locks in Java 9

# Agenda

Lambdas

Function, Predicate, Supplier, Optional

**BREAK**



Method references

Streams

String in Java 9

**BREAK**



StackWalker

Collection factory methods

Contended locks in Java 9

# Tests Configuration

## Hardware

- Intel i7-6700HQ Skylake
- 16GB DDR4 2133 MHz



## Software

- JDK 9.0.4 x64
- Java Measurement Harness
- perf
- Ubuntu 16.04.2





# Disclaimer

```
0x00007f92409afe83: mov     rax,0x0
0x00007f92409afe8d: lock   cmpxchg QWORD PTR [r10+0x16],r15
0x00007f92409afe93: jne     0x00007f92409afe9a
0x00007f92409afe95: or      eax,0x1
0x00007f92409afe98: jmp     0x00007f92409afeaa
0x00007f92409afe9a: test    eax,0x0
0x00007f92409afe9d: jne     0x00007f92409afecd
0x00007f92409afec0: mov     eax,0xc6
```

**Bytecode**

```
ALOAD 0
INVOKEDYNAMIC com/jpt/lambda/LambdaMain:Ljava/lang/String; [
// handle kind 0x7 : INVOKESPECIAL
java/lang/String;
    invokevirtual com/jpt/lambda/LambdaMain.lambda$capturingLambda_param$1(I)I,
    (I)I
]
ASTORE 1
```

**X86-64 Assembly Code**

# Lambdas

# Lambda Types

1

**Capturing Lambdas** - captures enclosing final (or effectively final) local variables, and/or fields from enclosing instances

```
int capturingLambda () {  
    Scalar s1;  
    s1 = () -> 11 * value;  
  
    return s1.get();  
}
```

# Lambda Types

1

**Capturing Lambdas** - captures enclosing final (or effectively final) local variables, and/or fields from enclosing instances

```
int capturingLambda () {  
    Scalar s1;  
    s1 = () -> 11 * value;  
  
    return s1.get();  
}
```

2

**Non-capturing Lambdas** - captures no state from its enclosing scope (i.e. *stateless* lambda)

```
int nonCapturingLambda () {  
    Scalar s1;  
    s1 = () -> 11 * 77;  
  
    return s1.get();  
}
```

# Capturing Lambda

```
@FunctionalInterface
public interface ScalarParam {
    int get(int x);
}

public class LambdaMain {

    int capturingLambda_param()
    {
        ScalarParam s1;
        s1 = (int x) -> x * value;

        return s1.get(11);
    }
}
```

```
public class LambdaMain {  
  
    capturingLambda_param() I {  
        ALOAD 0                                ; load this  
        INVOKEDYNAMIC get(LLambdaMain;) LScalarParam; [...]  
        ASTORE 1  
  
        ALOAD 1  
        BIPUSH 11                               ; push 11  
        INVOKEINTERFACE ScalarParam.get (I) I ; s1.get(11)  
        IRETURN  
    }  
  
    private synthetic lambda$capturingLambda_param$1(I) I {  
        ILOAD 1                                ; load var1  
        ALOAD 0                                ; load this  
        GETFIELD LambdaMain.value : I  
        IMUL                                    ; var1 * this.value  
        IRETURN  
    }  
}
```

# Bytecode

```
public class LambdaMain {
```

```
    capturingLambda_param() I {
```

```
        ALOAD 0                                ; load this
```

```
        INVOKEDYNAMIC get(LLambdaMain;) LScalarParam; [...]
```

```
        ASTORE 1
```

```
        ALOAD 1
```

```
        BIPUSH 11                               ; push 11
```

```
        INVOKEINTERFACE ScalarParam.get (I) I ; s1.get(11)
```

```
        IRETURN
```

```
    }
```

```
private synthetic lambda$capturingLambda_param$1(I) I {
```

```
    ILOAD 1                                ; load var1
```

```
    ALOAD 0                                ; load this
```

```
    GETFIELD LambdaMain.value : I
```

```
    IMUL                                    ; var1 * this.value
```

```
    IRETURN
```

```
}
```

```
}
```

synthetic  
method [\*]  
added by  
javac

[\*] synthetic method - a Java construct introduced by the compiler



# Bytecode

1

```
INVOKEDYNAMIC get(LLambdaMain;) LScalarParam; [  
  // handle kind 0x6 : INVOKESTATIC  
  java/lang/invoke/LambdaMetafactory.metafactory (  
    MethodHandles.Lookup caller;           "com.jtp.lambda.LambdaMain"  
    String invokedName;                     "get"  
    MethodType invokedType;                 "(LambdaMain) ScalarParam"  
    MethodType samMethodType;               "(int) int"  
    MethodHandle implMethod;                "MethodHandle (LambdaMain, int) int"  
    MethodType instantiatedMethodType;      "(int) int"  
  ) LCallSite;  
  
  // arguments:  
  (I) I,  
  // handle kind 0x7 : INVOKESPECIAL  
  LambdaMain.lambda$capturingLambda_param$1 (I) I,  
  (I) I  
]
```

2

```
INVOKINTERFACE ScalarParam.get (I) I
```

# What happens at runtime during **INVOKEDYNAMIC** call?

<< capturing-lambda case >>

# What happens at runtime during **INVOKEDYNAMIC** call?

**Short Answer:** **INVOKEDYNAMIC** returns an instance of the functional interface to which the lambda is being converted.

CallSite LambdaMetafactory.**metafactory()**

CallSite **buildCallSite()**

Class<?> innerClass = **spinInnerClass()**

*/\*\* generates a class file which implements the functional interface \*/*  
**LambdaMain\$\$Lambda\$49.class**

*/\*\* if requested, dump out to a file for debugging purposes \*/*  
**-Djdk.internal.lambda.dumpProxyClasses=<path>**

*/\*\* creates and returns a call site having the target Method Handle a reference to a proxy of a inner generated class \*/*  
**MH ← LambdaMain\$\$Lambda\$49/1864350231.get\$Lambda()**  
**return callSite ← new ConstantCallSite(MH)**

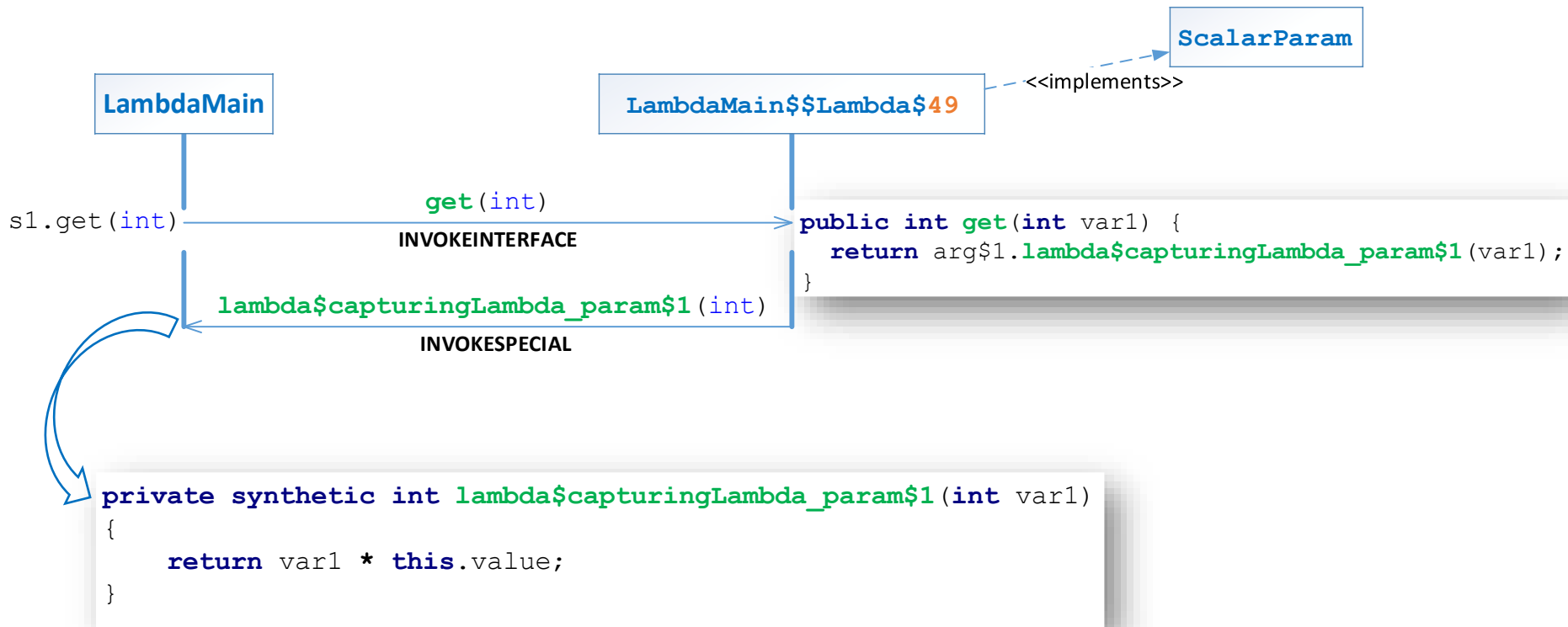
*/\*\* links invokedynamic to target method implementation which basically returns an instance of the functional interface (~ lambda object) \*/*

**callSite.dynamicInvoker()**  
**Invokers.linkToTargetMethod(type)**

# What happens during **INVOKEINTERFACE** call?

<< capturing-lambda case >>

# INVOKEINTERFACE ScalarParam.get (I)



**How does the auto-generated  
LambdaMain\$\$Lambda\$49 class look?**

<< capturing-lambda case >>

# -Djdk.internal.lambda.dumpProxyClasses=<path>

```
// $FF: synthetic class
final class LambdaMain$$Lambda$49 implements ScalarParam {
    private final LambdaMain arg$1;

    private LambdaMain$$Lambda$49(LambdaMain var1) {
        this.arg$1 = var1;
    }

    private static ScalarParam get$Lambda(LambdaMain var0) {
        return new LambdaMain$$Lambda$49(var0);
    }

    @Hidden
    public int get(int var1) {
        return this.arg$1.lambda$capturingLambda_param$1(var1);
    }
}
```



# -Djdk.internal.lambda.dumpProxyClasses=<path>

Global LambdaMetafactory counter used to generate unique lambda class names



```
// $FF: synthetic class
final class LambdaMain$$Lambda$49 implements ScalarParam {
    private final LambdaMain arg$1;

    private LambdaMain$$Lambda$49(LambdaMain var1) {
        this.arg$1 = var1;
    }

    private static ScalarParam get$Lambda(LambdaMain var0) {
        return new LambdaMain$$Lambda$49(var0);
    }

    @Hidden
    public int get(int var1) {
        return this.arg$1.lambda$capturingLambda_param$1(var1);
    }
}
```

# -Djdk.internal.lambda.dumpProxyClasses=<path>

Global LambdaMetafactory counter used to generate unique lambda class names

```
// $FF: synthetic class
final class LambdaMain$$Lambda$49 implements ScalarParam {
    private final LambdaMain arg$1;

    private LambdaMain$$Lambda$49(LambdaMain var1) {
        this.arg$1 = var1;
    }

    private static ScalarParam get$Lambda(LambdaMain var0) {
        return new LambdaMain$$Lambda$49(var0);
    }

    @Hidden
    public int get(int var1) {
        return this.arg$1.lambda$capturingLambda_param$1(var1);
    }
}
```

Each call creates a new instance

# -Djdk.internal.lambda.dumpProxyClasses=<path>

Global LambdaMetafactory counter used to generate unique lambda class names

```
// $FF: synthetic class
final class LambdaMain$$Lambda$49 implements ScalarParam {
    private final LambdaMain arg$1;

    private LambdaMain$$Lambda$49(LambdaMain var1) {
        this.arg$1 = var1;
    }

    private static ScalarParam get$Lambda(LambdaMain var0) {
        return new LambdaMain$$Lambda$49(var0);
    }

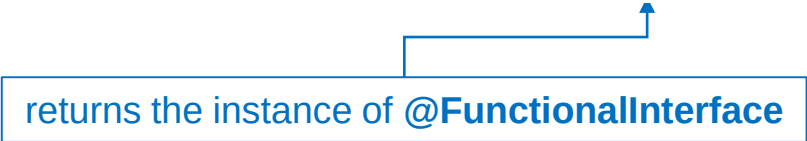
    @Hidden
    public int get(int var1) {
        return this.arg$1.lambda$capturingLambda_param$1(var1);
    }
}
```

Each call creates a new instance

Calls synthetic method defined in LambdaMain

# Capturing Lambdas

INVOKEDYNAMIC ← CallSite( MH(LambdaMain\$\$Lambda\$49/1864350231.get\$Lambda()) )



returns the instance of @FunctionalInterface

LambdaMetafactory.**metafactory()** only invoked once per capture site

Any successive call on the same **call site** uses the same proxy (i.e. **get\$Lambda()**)  
of generated anonymous class

**CallSite** is lazily linked, so factory sites that are never invoked are never linked

# Performance Costs

|                       | Linkage Cost                                        | Capture Cost                  | Invocation Cost                              |
|-----------------------|-----------------------------------------------------|-------------------------------|----------------------------------------------|
| Capturing Lambdas     | set the lambda capture<br>- one time per callSite - | create lambda instance        | invoke lambda method<br><b>INVOKESPECIAL</b> |
| Non-capturing Lambdas | set the lambda capture<br>- one time per callSite - | no need for any<br>instance ! | invoke lambda method<br><b>INVOKESTATIC</b>  |
| Inner classes         | load the class                                      | invoke the constructor        | invoke the method<br><b>INVOKESPECIAL</b>    |

**SUM UP:** “Worst case (i.e. capturing lambda) as inner classes, best case (i.e. non-capturing lambda) better!” – Brian Goetz

# Non-capturing Lambdas

```
// $FF: synthetic class
final class LambdaMain$$Lambda$49 implements ScalarParam {
    private LambdaMain$$Lambda$49() {

    }

    @Hidden
    public int get(int var1) {
        return LambdaMain.lambda$nonCapturingLambda_param$1(var1);
    }
}
```

calls synthetic static method from LambdaMain

No need to create any instance! No **get\$Lambda()** method needed!

# Lambda Stack Frames

```
int capturingLambda_param_exception()
{
    ScalarParam s1;
    s1 = (int x) -> {
        if (10 > 5) // stupid trick, bypass the IDE check
            throw new RuntimeException("Bucharest Java User Group 2018 Exception");
        return x * value;
    };

    return s1.get(11);
}
```

```
Exception in thread "main" java.lang.RuntimeException: Bucharest Java User Group 2018 Exception
    at com.jpt.lambda.LambdaMain.lambda$capturingLambda_param_exception$2(LambdaMain.java:28)
    at com.jpt.lambda.LambdaMain.capturingLambda_param_exception(LambdaMain.java:32)
    at com.jpt.lambda.LambdaMain.main(LambdaMain.java:58)
```

?

Why **LambdaMain\$\$Lambda\$49** is not printed in the stack trace?



# Solution

## Launch JVM with:

`-XX:+UnlockDiagnosticVMOptions` `-XX:+ShowHiddenFrames`

```
Exception in thread "main" java.lang.RuntimeException: Bucharest Java User Group 2018 Exception
    at com.jpt.lambda.LambdaMain.lambda$capturingLambda_param_exception$2(LambdaMain.java:28)
    at com.jpt.lambda.LambdaMain$$Lambda$49/710714889.get(<Unknown>:1000005)
    at com.jpt.lambda.LambdaMain.capturingLambda_param_exception(LambdaMain.java:32)
    at com.jpt.lambda.LambdaMain.main(LambdaMain.java:58)
```

`-XX:+ShowHiddenFrames` includes hidden stack frames (e.g. lambda generated)

# Benchmarks

**Capturing Lambda**

**VS.**

**Non-Capturing Lambda**

**VS.**

**Capturing Anonymous Class**

**VS.**

**Non-Capturing Anonymous Class**

# Capturing Lambda

```
@Benchmark
public int capturingLambda() {
    Scalar s1;

    s1 = () -> 11 * value;

    return s1.get();
}
```

# Capturing Lambda

```
@Benchmark
public int capturingLambda() {
    Scalar s1;

    s1 = () -> 11 * value;

    return s1.get();
}
```



explicit memory load  
for captured field

**capturingLambda**

```
mov     r11d, DWORD PTR [r10+0x70]
        ;*getstatic value
imul    eax, DWORD PTR [r12+r11*8+0xc], 0xb
        ;*imul
```

# Capturing Lambda

```
@Benchmark
public int capturingLambda() {
    Scalar s1;

    s1 = () -> 11 * value;

    return s1.get();
}
```



explicit memory load  
for captured field

**capturingLambda**

```
mov     r11d, DWORD PTR [r10+0x70]
        ;*getstatic value
imul    eax, DWORD PTR [r12+r11*8+0xc], 0xb
        ;*imul
```

no INVOKEDYNAMIC  
callee inlined

# Non-Capturing Lambda

```
@Benchmark
public int nonCapturingLambda() {
    Scalar s1;

    s1 = () -> 11 * 77;

    return s1.get();
}
```

# Non-Capturing Lambda

```
@Benchmark
public int nonCapturingLambda() {
    Scalar s1;

    s1 = () -> 11 * 77;

    return s1.get();
}
```



**nonCapturingLambda**

```
mov     eax, 0x34f
        ; 11*77
```

constant folded



# Non-Capturing Lambda

```
@Benchmark
public int nonCapturingLambda() {
    Scalar s1;

    s1 = () -> 11 * 77;

    return s1.get();
}
```



no INVOKEDYNAMIC  
callee inlined

**nonCapturingLambda**

```
mov     eax, 0x34f
        ; 11*77
```

constant folded

# Capturing Anonymous Class

```
@Benchmark
public int capturingAnonymousClass() {
    Scalar s1;

    s1 = new Scalar() {
        @Override
        public int get() {
            return 11 * value;
        }
    };
    return s1.get();
}
```

# Capturing Anonymous Class

```
@Benchmark
public int capturingAnonymousClass() {
    Scalar s1;

    s1 = new Scalar() {
        @Override
        public int get() {
            return 11 * value;
        }
    };
    return s1.get();
}
```



similar to  
capturing lambda

**capturingAnonymousClass**

```
mov     r11d,DWORD PTR [r10+0x70]
        ;*getstatic value
imul    eax,DWORD PTR [r12+r11*8+0xc],0xb
        ;*imul
```

# Non-Capturing Anonymous Class

```
@Benchmark
public int nonCapturingAnonymousClass ()
    Scalar s1;

    s1 = new Scalar() {
        @Override
        public int get() {
            return 11 * 77;
        }
    };
    return s1.get();
}
```

# Non-Capturing Anonymous Class

```
@Benchmark
public int nonCapturingAnonymousClass()
{
    Scalar s1;

    s1 = new Scalar() {
        @Override
        public int get() {
            return 11 * 77;
        }
    };
    return s1.get();
}
```



```
nonCapturingAnonymousClass
    mov     eax, 0x34f
           ; 11*77
```

similar to  
non capturing lambda

| Case                            | ns/op |
|---------------------------------|-------|
| Capturing Lambda                | 2,513 |
| Non-capturing Lambda            | 2,122 |
| Capturing Anonymous Classes     | 2,497 |
| Non-capturing Anonymous Classes | 2,211 |

In all cases: **no INVOKEDYNAMIC**; no object allocation; callee inlined

Capturing cases: **pay the cost of memory loads** (i.e. captured field)

# Chaining Lambdas

$() \rightarrow () \rightarrow \dots \rightarrow () \rightarrow \text{scalar}$

```
@Benchmark
// () -> () -> 111
public int depth1() {
    Level1 l_1;
    Level0 l_0;

    l_0 = () -> 111;
    l_1 = () -> l_0;

    return l_1.next().get();
}
```

```
@Benchmark
// () -> () -> () -> 111
public int depth2() {
    Level2 l_2;
    Level1 l_1;
    Level0 l_0;

    l_0 = () -> 111;
    l_1 = () -> l_0;
    l_2 = () -> l_1;

    return l_2.next().next().get();
}
```



```

@Benchmark
// () -> () -> 111
public int depth1() {
    Level1 l_1;
    Level0 l_0;

    l_0 = () -> 111;
    l_1 = () -> l_0;

    return l_1.next().get();
}

```

**depth1**

```

mov     eax,0x6f
        ; 111

```



```

@Benchmark
// () -> () -> () -> 111
public int depth2() {
    Level2 l_2;
    Level1 l_1;
    Level0 l_0;

    l_0 = () -> 111;
    l_1 = () -> l_0;
    l_2 = () -> l_1;

    return l_2.next().next().get();
}

```

**depth2**

```

mov     eax,0x6f
        ; 111

```

```

@Benchmark
// () -> () -> () -> () -> 111
public int depth3() {
    Level3 l_3;
    Level2 l_2;
    Level1 l_1;
    Level0 l_0;

    l_0 = () -> 111;
    l_1 = () -> l_0;
    l_2 = () -> l_1;
    l_3 = () -> l_2;

    return l_3.next().next()
               .next().get();
}

```

?

Will it be optimized in the same manner as **depth1** and **depth2**?

### depth3

```
mov     QWORD PTR [r15+0x388],r10
; *invokespecial <init>;
; - LambdaJmh$$Lambda$51/389994213::get$Lambda@5

mov     r11d,DWORD PTR [rbx+0x8]
cmp     r11d,0xf8023285      ; {metadata(&apos;LambdaJmh$$Lambda$50&apos;;)}
jne     0x00007fcb856441d0   ; *invokeinterface next
mov     ebp,DWORD PTR [rbx+0xc] ; *getfield arg$1
; - LambdaJmh$$Lambda$50/588571575::next@1

mov     r10d,DWORD PTR [r12+rbp*8+0x8]
cmp     r10d,0xf8023205      ; {metadata(&apos;LambdaJmh$$Lambda$49&apos;;)}
jne     0x00007fcb856441e5
lea     r10,[r12+rbp*8]      ; *invokeinterface next
mov     ebp,DWORD PTR [r10+0xc] ; *getfield arg$1
; - LambdaJmh$$Lambda$49/145604150::next@1

mov     r10d,DWORD PTR [r12+rbp*8+0x8]
cmp     r10d,0xf8023185      ; {metadata(&apos;LambdaJmh$$Lambda$48&apos;;)}
ne      0x00007fcb856441f5   ; *invokeinterface get

mov     eax,0x6f             ; 111
```

### depth3

```
mov     QWORD PTR [r15+0x388],r10
; *invokespecial <init>;
; - LambdaJmh$$Lambda$51/389994213::get$Lambda@5

I_2 {
mov     r11d,DWORD PTR [rbx+0x8]
cmp     r11d,0xf8023285      ; {metadata(&apos;LambdaJmh$$Lambda$50&apos;;)}
jne     0x00007fcb856441d0    ; *invokeinterface next
mov     ebp,DWORD PTR [rbx+0xc] ; *getfield arg$1
; - LambdaJmh$$Lambda$50/588571575::next@1

I_1 {
mov     r10d,DWORD PTR [r12+rbp*8+0x8]
cmp     r10d,0xf8023205      ; {metadata(&apos;LambdaJmh$$Lambda$49&apos;;)}
jne     0x00007fcb856441e5
lea     r10,[r12+rbp*8]      ; *invokeinterface next
mov     ebp,DWORD PTR [r10+0xc] ; *getfield arg$1
; - LambdaJmh$$Lambda$49/145604150::next@1

I_0 {
mov     r10d,DWORD PTR [r12+rbp*8+0x8]
cmp     r10d,0xf8023185      ; {metadata(&apos;LambdaJmh$$Lambda$48&apos;;)}
ne      0x00007fcb856441f5    ; *invokeinterface get

mov     eax,0x6f              ; 111
```

depth3

mov QWORD PTR [r15+0x388],r10

Useless

; \*invokespecial <init>;  
; - LambdaJmh\$\$Lambda\$51/389994213::get\$Lambda@5

I\_2

mov r11d,DWORD PTR [rbx+0x8]  
cmp r11d,0xf8023285  
jne 0x00007fcb856441d0  
mov ebp,DWORD PTR [rbx+0xc]

Useless

; {metadata(&apos;LambdaJmh\$\$Lambda\$50&apos;)}  
; \*invokeinterface next  
; \*getfield arg\$1  
; - LambdaJmh\$\$Lambda\$50/588571575::next@1

I\_1

mov r10d,DWORD PTR [r12+rbp\*8+0x8]  
cmp r10d,0xf8023205  
jne 0x00007fcb856441e5  
lea r10,[r12+rbp\*8]  
mov ebp,DWORD PTR [r10+0xc]

Useless

; {metadata(&apos;LambdaJmh\$\$Lambda\$49&apos;)}  
; \*invokeinterface next  
; \*getfield arg\$1  
; - LambdaJmh\$\$Lambda\$49/145604150::

OUCH!



I\_0

mov r10d,DWORD PTR [r12+rbp\*8+0x8]  
cmp r10d,0xf8023185  
jne 0x00007fcb856441f5

Useless

; {metadata(&apos;LambdaJmh\$\$Lambda\$48&apos;)}  
; \*invokeinterface get

mov eax,0x6f

; 111

| Case    | ns/op | GC count |
|---------|-------|----------|
| depth1  | 2,2   | 0        |
| depth2  | 4,6   | 185      |
| depth3  | 6,3   | 231      |
| depth4  | 9,1   | 236      |
| depth10 | 29,0  | 246      |

Time per iteration is proportional to chain depth (i.e. lambda **depth** )

There is also a GC impact since JIT cannot optimize all depth cases, leading to object heap allocations (i.e. **get\$Lambda()** calls)

# LAMBDA TAKE AWAY

For capturing cases, lambdas might create a lot of objects on the heap (i.e. same as anonymous classes)

Do not really abuse of lambda chain calls since JIT can struggle in such cases

Use `-Djdk.internal.lambda.dumpProxyClasses` to dump anonymous generated classes for further analysis

Use `-XX:+ShowHiddenFrames` to have the full stack trace

Use `-XX:+PrintInlining` and `-XX:BCEATraceLevel=3` to check for possible inlining or escape analysis issues

# Function, Predicate, Supplier, Optional



# Function<T, R>

<< a function that accepts one argument **T** and produces a result **R** >>

```
@Param({"42"})
public static Integer intValue;

@Benchmark
public static Integer function_int() {
    Function<Integer, Integer> func = (in) -> in << 2;
    return func.apply(intValue);
}

@Benchmark
public static Integer baseline_int() {
    return intValue << 2;
}
```

# Function<T,R> Bytecode

```
public static Integer function_int()  
    INVOKEDYNAMIC (  
        ( MH(metaFactory),  
          MH(INVOKEINTERFACE java/util/function/Function.apply),  
          MH(INVOKESTATIC FunctionJmh.lambda$function_int$0(LInteger;)LInteger)  
            ( intValue )  
        )  
    )  
    INVOKEINTERFACE java/util/function/Function.apply (LObject;)LObject;  
  
private static synthetic lambda$function_int$0(LInteger;)LInteger;  
    ALOAD 0  
    INVOKEVIRTUAL java/lang/Integer.intValue ()I  
    ICONST_2  
    ISHL  
    INVOKESTATIC java/lang/Integer.valueOf (I)LInteger;
```

Desugared the same way as lambdas + interface method calls

&lt;&lt;implements&gt;&gt;

FunctionJmh

FunctionJmh\$\$Lambda\$49

apply(Object)

INVOKEINTERFACE

lambda\$function\_int\$0(Integer)

INVOKESTATIC

```
public Object apply(Object var1) {  
    return  
        FunctionJmh.lambda$function_int$0((Integer)var1);  
}
```

```
private static synthetic Integer lambda$function_int$0(Integer var1)  
{  
    return var1 << 2;  
}
```

# Function<T,R> Assembly

```
public static Integer function_int()
    mov     r11d,DWORD PTR [r10+0x74]
           ;*getstatic intValue
           ; - FunctionJmh::function_int@7 (line 64)
    mov     ebp,DWORD PTR [r12+r11*8+0xc]
    shl     ebp,0x2           ;*ishl
           ; - FunctionJmh::lambda$function_int$0@5 (line 63)
    mov     DWORD PTR [rax+0xc],ebp ;*synchronization entry
           ; - java.lang.Integer::valueOf@-1 (line 1048)
```

**baseline\_int** is optimized almost the same way of **function\_int**!

|              | ns/op |
|--------------|-------|
| function_int | 4,6   |
| baseline_int | 4,5   |

Equivalent optimizations at assembly level; no difference in performance!

# Predicate<T>

<< a boolean valued function of one argument **T** >>

```
@Param({"42"})
static Integer intValue;

@Benchmark
public static boolean predicate_int() {
    Predicate<Integer> predicate = (i) -> (i << 2) > 0;
    return predicate.test(intValue);
}

@Benchmark
public static boolean baseline_int() {
    return (intValue << 2) > 0;
}
```



# Predicate<T> Assembly

```
public static predicate_int() Z
    mov     r10d,DWORD PTR [r10+0x70]
           ;*getstatic intValue
           ; - PredicateJmh::predicate_int@7 (line 46)
    mov     r10d,DWORD PTR [r12+r10*8+0xc]
           ; implicit exception: dispatches to 0x00007f8cf56401ed
    shl     r10d,0x2
           ;*ishl
           ; - PredicateJmh::lambda$predicate_int$0@5 (line 45)
    mov     eax,0x1
           ;*returns true
```

**baseline\_int** is optimized almost the same way of **predicate\_int**!

|               | ns/op |
|---------------|-------|
| predicate_int | 2,6   |
| baseline_int  | 2,6   |

Equivalent optimizations at assembly level; no difference in performance!

# Supplier<T>

<< a supplier of results of type **T** >>

```
@Param({"42"})
static Integer intValue;

@Benchmark
public static Integer supplier_integer() {
    Supplier<Integer> supplier = () -> intValue;
    return supplier.get();
}

@Benchmark
public static Integer baseline_integer() {
    return intValue;
}
```

# Supplier<T> Assembly

```
public static supplier_int()Ljava/lang/Integer;
    movabs r10,0x6d71b47c8 ; {oop(&apos;SupplierJmh&apos;)}
    mov     r11d,DWORD PTR [r10+0x70]
    mov     rax,r11          ;*getstatic intValue
                                ; - SupplierJmh::lambda$supplier_int$0@0 (line 58)
```

**baseline\_int** is optimized almost the same way of **supplier\_int**!

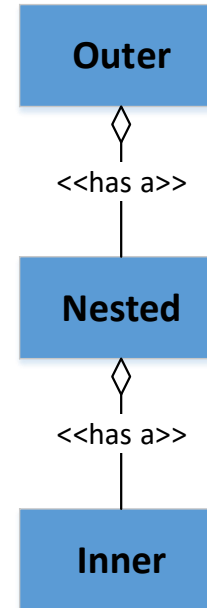
|              | ns/op |
|--------------|-------|
| supplier_int | 2,6   |
| baseline_int | 2,5   |

Equivalent optimizations at assembly level; no difference in performance!

# Optional<T>

<< a container object which may or may not contain a non-null value. If a value is present, *isPresent()* returns true and *get()* returns the value **T** >>

```
public class Outer {  
  
    public Nested nested;  
    public Outer(Integer value) {  
        this.nested = new Nested(value);  
    }  
}  
  
public class Nested {  
  
    public Inner inner;  
    public Nested(Integer value) {  
        this.inner = new Inner(value);  
    }  
}  
  
public class Inner {  
  
    public Integer value;  
    public Inner(Integer value) {  
        this.value = value;  
    }  
}
```





```

@Param({"77"})
public static Integer intValue;

static Outer outer = new Outer(intValue);

@Benchmark
public static int optionalChain() {
    return Optional.of(outer)
        .flatMap(o -> Optional.ofNullable(o.nested))
        .flatMap(n -> Optional.ofNullable(n.inner))
        .flatMap(i -> Optional.ofNullable(i.value))
        .get();
}

@Benchmark
public static int ifComparison() {
    return ( outer != null &&
            outer.nested != null &&
            outer.nested.inner != null
            )
        ? outer.nested.inner.value : null; // returns intValue
}

```

|               | ns/op |
|---------------|-------|
| ifComparison  | 3,3   |
| optionalChain | 3,8   |

?

Can you guess why the **ifComparison** is slightly better?

```
public static int ifComparison()
```

```
    mov     r10d,DWORD PTR [r10+0x74]
           ;*getstatic outer
    mov     r11d,DWORD PTR [r12+r10*8+0xc]
           ;*getfield nested
           ; implicit exception: dispatches to 0x00007f074563ebe9
    mov     ebp,DWORD PTR [r12+r11*8+0xc]
           ;*getfield inner
           ; implicit exception: dispatches to 0x00007f074563ebfd
    mov     r10d,DWORD PTR [r12+rbp*8+0xc]
           ;*getfield value
           ; implicit exception: dispatches to 0x00007f074563ec11
    mov     eax,DWORD PTR [r12+r10*8+0xc]
           ;*getfield intValue
           ; - java.lang.Integer::intValue@1 (line 1123)
           ; implicit exception: dispatches to 0x00007f074563ebda
```

**NULL** checks relies on signals (e.g. SEG\_FAULT) natively provided by hardware and handled by OS to optimize hot paths.

```
public static int optionalChain()
```

```
    mov     r10d,DWORD PTR [r10+0x74]
                                           ;*getstatic outer

    test    r10d,r10d
    je      0x00007fcc1163ee70           ;*ifnonnull
    shl     r10,0x3                      ;*checkcast
    mov     ebp,DWORD PTR [r10+0xc]

                                           ;*getfield nested
    test    ebp,ebp
    je      0x00007fcc1163ee85           ;*ifnonnull
    lea     r10,[r12+rbp*8]              ;*checkcast
    mov     ebp,DWORD PTR [r10+0xc]      ;*getfield inner

    test    ebp,ebp
    je      0x00007fcc1163ee95           ;*ifnonnull
    lea     r10,[r12+rbp*8]              ;*checkcast
    mov     ebp,DWORD PTR [r10+0xc]      ;*getfield value
    mov     eax,DWORD PTR [r12+rbp*8+0xc]
                                           ;*getfield intValue
```

Explicit **NULL** checks and casts!

**BREAK**

# Method Reference

**A `method reference` is a more compact and easy-to-read construct to replace a lambda expression.**

# Kinds of Method References

|                                                             |                                 |
|-------------------------------------------------------------|---------------------------------|
| Reference to a static method                                | <b>Class::staticMethod</b>      |
| Reference to an instance method of a particular object      | <b>instance::instanceMethod</b> |
| Reference to an instance method of an arbitrary object type | <b>Type::method</b>             |
| Reference to a constructor                                  | <b>ClassName::new</b>           |



# instance::instanceMethod

```
public class ArithmeticOperation {  
  
    public Integer multiply(Integer n){  
        return n << 2;  
    }  
}
```

```
public static ArithmeticOperation numInstance = new ArithmeticOperation();
```

@Benchmark

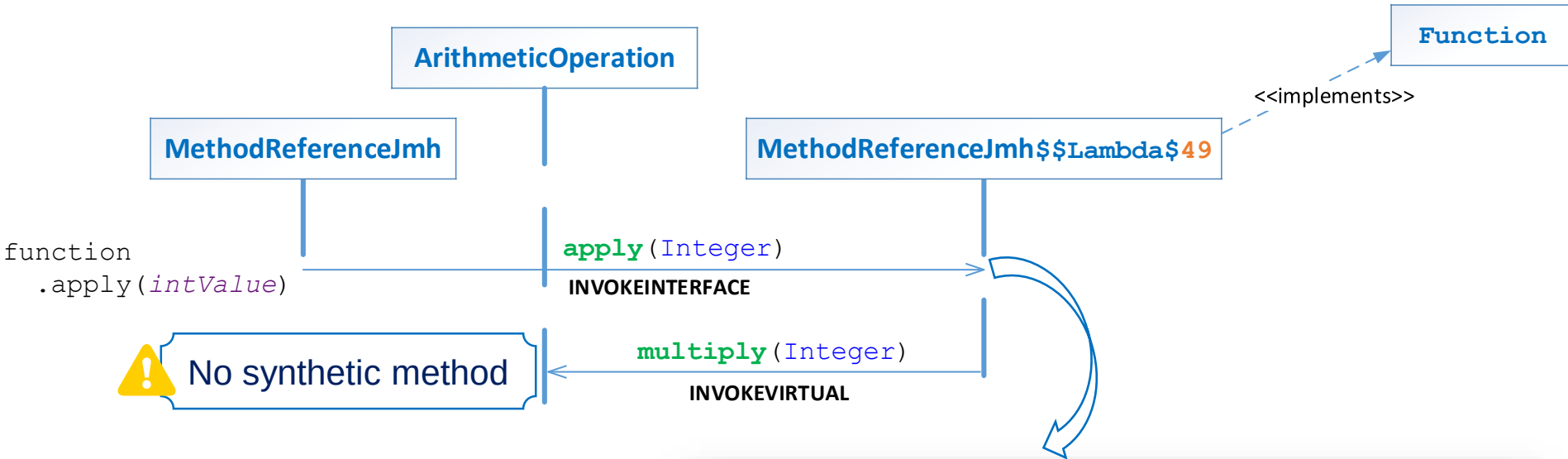
```
public static Integer boundedInstance() {  
    Function<Integer, Integer> function = numInstance::multiply;  
    // ... = (in) -> numInstance.multiply(in); LAMBDA version  
    return function.apply(intValue);  
}
```

# instance::instanceMethod

```
public static boundedInstance()Ljava/lang/Integer;
  INVOKEDYNAMIC (
    ( MH(metaFactory),
      MH(INVOKEINTERFACE java/util/function/Function.apply),
      MH(INVOKEVIRTUAL ArithmeticOperation.multiply(LInteger;)LInteger
        ( intValue )
      )
    )
  )
  INVOKEINTERFACE java/util/function/Function.apply (LObject;)LObject;
```



There is no synthetic method generated!  
A constant MH for referenced method is loaded (e.g. **multiply()**)



```
private final ArithmeticOperation arg$1;

public Object apply(Object var1) {
    return this.arg$1.multiply((Integer) var1);
}
```

---

|                        | <b>ns/op</b> |
|------------------------|--------------|
| boundedInstance        | 4,31         |
| invokeVirtual_baseline | 4,30         |

---

# Class::new

```
@Benchmark
public static ArithmeticOperation constructorReference() {
    Supplier<ArithmeticOperation> supplier = ArithmeticOperation::new;
    // ... = () -> new ArithmeticOperation(); LAMBDA version
    return supplier.get();
}
```

# Class::new

```
@Benchmark
public static ArithmeticOperation constructorReference() {
    Supplier<ArithmeticOperation> supplier = ArithmeticOperation::new;
    // ... = () -> new ArithmeticOperation(); LAMBDA version
    return supplier.get();
}
```



```
public static constructorReference() Lcom/jpt/function/ArithmeticOperation;
    INVOKEDYNAMIC (
        ( MH(metaFactory),
          MH(INVOKEINTERFACE java/util/function/Supplier.get),
          MH(NEWINVOKESPECIAL ArithmeticOperation.<init>()) V
        )
    )
)
```



Equivalent to **new Class()** call!

---

|                      | <b>ns/op</b> |
|----------------------|--------------|
| constructorReference | 3,7          |
| constructor_baseline | 3,6          |

---

instead of using

**AN ANONYMOUS CLASS**

you can use

**A LAMBDA EXPRESSION**

but if this just calls one method you can use

**A METHOD REFERENCE**



# INVOKEDYNAMIC - Golden Hammer ☺



# Streams



| Stream structure    | Characteristics     | API examples                         |
|---------------------|---------------------|--------------------------------------|
| <b>SOURCE</b>       | collections; arrays | stream(); parallelStream(); range(); |
| <b>INTERMEDIATE</b> | stateless           | filter(); map(); flatMap();          |
|                     | statefull           | sorted(); limit(); distinct();       |
| <b>TERMINAL</b>     | short-circuit       | findAny(); findFirst(); anyMatch();  |
|                     | non short-circuit   | reduce(); collect(); forEach();      |

**Lazy\*** - intermediate operations are not triggered unless a terminal operation is present!

True for parallel streams; false otherwise

```
public static <T> Stream<T> stream(Spliterator<T> spliterator, boolean parallel)
```

Access the elements and possibly decomposes the input source for parallel execution (splitting)

True for parallel streams; false otherwise

```
public static <T> Stream<T> stream(Splitterator<T> splitterator, boolean parallel)
```

Access the elements and possibly decomposes the input source for parallel execution (splitting)

```
public interface java.util.Splitterator<T> {  
  
    /** process a single element */  
    boolean tryAdvance(Consumer<? super T> action);  
  
    /** process all remaining elements */  
    default void forEachRemaining(Consumer<? super T> action) {  
        do { } while (tryAdvance(action));  
    }  
  
    /** split remaining elements. Used for parallel processing */  
    Splitterator<T> trySplit();  
  
}
```

True for parallel streams; false otherwise

```
public static <T> Stream<T> stream(Splitterator<T> splitterator, boolean parallel)
```

Access the elements and possibly decomposes the input source for parallel execution (splitting)

```
public interface java.util.Splitterator<T> {
```

```
    /** process a single element */
```

```
    boolean tryAdvance(Consumer<? super T> action);
```

short-circuit

```
    /** process all remaining elements */
```

```
    default void forEachRemaining(Consumer<? super T> action) {
```

non short-circuit

```
        do { } while (tryAdvance(action));
```

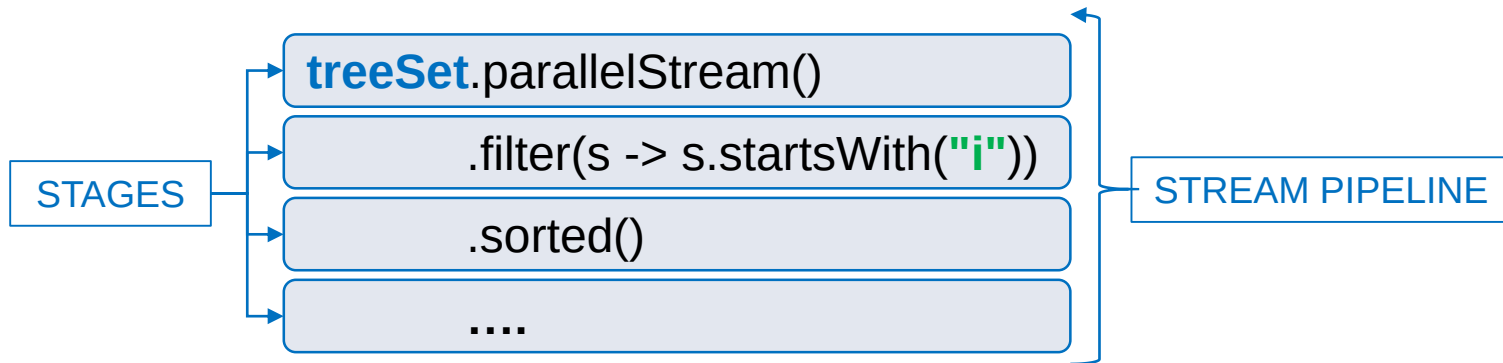
```
    }
```

```
    /** split remaining elements. Used for parallel processing */
```

```
    Splitterator<T> trySplit();
```

```
}
```

```
TreeSet<String> treeSet = new TreeSet<>();
```



**STREAM PIPELINE** = SOURCE + {INTERMEDIATE OPERATIONS}

Each pipeline **STAGE** is described by a bitmap of ***stream flags***

## Stream Flags - used internally for optimization purposes



Each **STAGE** could add, remove or preserve flags!

```
enum java.util.stream.StreamOpFlag {  
  
    static int combineOpFlags(int newStreamOrOpFlags, int prevCombOpFlags) {  
        // 0x01 or 0x10 nibbles are transformed to 0x11  
        // 0x00 nibbles remain unchanged  
        // Then all the bits are flipped  
        // Then the result is logically or'ed with the operation flags.  
        return (prevCombOpFlags & StreamOpFlag.getMask(newStreamOrOpFlags)) | newStreamOrOpFlags;  
    }  
}
```



# Stream Flags - Intermediate Operations

| ORDERED | SORTED | DISTINCT | SIZED |                 |
|---------|--------|----------|-------|-----------------|
| ×       | ×      | ×        | —     | filter()        |
| ×       | —      | —        | ×     | map()           |
| +       | +      | ×        | ×     | sorted()        |
| ×       | ×      | ×        | —     | limit(); skip() |
| ×       | ×      | +        | —     | distinct()      |
| —       | ×      | ×        | ×     | unordered()     |



preserve



add



remove

# Stream Flags - Example

```
TreeSet<String> treeSet = new TreeSet<>();
```

```
treeSet.parallelStream()
```

**DISTINCT | SORTED | ORDERED**

```
static final class java.util.KeySpliterator<K,V>
    public int characteristics() {
        return (side == 0 ? Spliterator.SIZED : 0) |
            Spliterator.DISTINCT | Spliterator.SORTED | Spliterator.ORDERED;
    }
}
```

Stream inherited flags\*

```
.filter(s -> s.startsWith("i"))
```

**+ NOT\_SIZED**

```
.sorted()
```

**+ IS\_ORDERED | + IS\_SORTED**

```
....
```

...

**flags\*** - each Stream type has defined Spliterator characteristics.

# Redundant Intermediate Operations

```
TreeSet<String> treeSet = new TreeSet<>();
```

```
@Benchmark
```

```
public Object[] treeSet_explicit_sort() {  
    Object[] result = treeSet.parallelStream()  
        .filter(s -> s.startsWith("i"))  
        .sorted() // sorting again !?  
        .toArray();  
  
    return result;  
}
```

← implicitly sorted

?

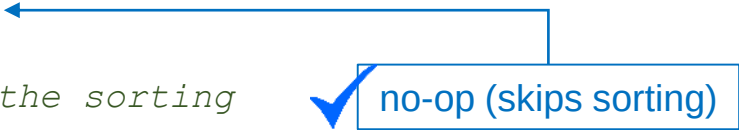
What happens if **sorted** is explicitly called?

# Solution

**sorted()** is actually skipped 😊

```
final class java.util.stream.SortedOps
```

```
public Sink<T> opWrapSink(int flags, Sink<T> sink) {  
    // If the input is already naturally sorted and this operation  
    // also naturally sorted then this is a no-op  
    if (StreamOpFlag.SORTED.isKnown(flags) && isNaturalSort)  
        return sink;  
    }  
    // else trigger the sorting  
}
```



*"In some cases, Streams can use knowledge of the source and preceding operations to elide an operation entirely." – Brian Goetz*

# Redundant Intermediate Operations

```
TreeSet<String> treeSet = new TreeSet<> (Comparator.naturalOrder() );
```

```
@Benchmark
```

```
public Object[] treeSet_explicit_comparator_sort() {  
    Object[] result = treeSet.parallelStream()  
        .filter(s -> s.startsWith("i"))  
        .sorted()  
        .toArray();  
  
    return result;  
}
```

explicit Comparator

?

Do you think **sorted** is executed this time?

# Solution

**sorted()** is not a no-op anymore 😞

```
final class java.util.stream.StreamOpFlag
```

```
static int fromCharacteristics(Spliterator<?> spliterator) {  
    int characteristics = spliterator.characteristics();  
    if ((characteristics & Spliterator.SORTED) != 0 && spliterator.getComparator() != null) {  
        // Do not propagate the SORTED characteristic if it does not correspond  
        // to a natural sort order  
        return characteristics & SPLITERATOR_CHARACTERISTICS_MASK & ~Spliterator.SORTED;  
    }  
    //...  
}
```

clear SORTED flag



Providing ANY Comparator on a sorted Stream clears **SORTED** flag!  
This might lead to redundant sorting operations!

| <b>TreeSet – parallel stream</b> | <b>us/op</b> |
|----------------------------------|--------------|
| implicit sort                    | 70           |
| explicit sort                    | 74           |
| explicit comparator sort         | 79           |

**Explicit sort** still have a bit of penalty for no-op in comparison to **implicit sort**

**Explicit comparator sort** forces to sort again an already sorted stream !

# Stream<T>

VS.

## IntStream, LongStream, DoubleStream



Use dedicated Stream classes for primitives (e.g. int, long or double) to avoid unnecessary boxing unboxing operations!



```
@Param({ "10_000" })
```

```
int arraySize;
```

```
Integer[] integerList;    // initialized with 0, 1, 2 ... 10_000
```

```
@Benchmark
```

```
public int intStream_range() {
```

```
    return IntStream.range(0, arraySize).parallel().sum();
```

```
}
```

```
@Benchmark
```

```
public int arrays_stream_integer_list() {
```

```
    return Arrays.stream(integerList).parallel().mapToInt(n -> n).sum();
```

```
}
```

```
@Benchmark
```

```
public int stream_of_integer_list() {
```

```
    return Stream.of(integerList).parallel().mapToInt(n -> n).sum();
```

```
}
```

---

|                            | <b>us/op</b> |
|----------------------------|--------------|
| intStream_range            | 16,5         |
| arrays_stream_integer_list | 18,9         |
| stream_of_integer_list     | 19,2         |

**IntStream** performs better than **Stream<Integer>** approaches



What is the difference between these two calls?

**Stream.of([]) vs. Arrays.stream([])**



What is the difference between these two calls?

## Stream.of([]) vs. Arrays.stream([])

```
public interface Stream<T> extends BaseStream<T, Stream<T>> {  
    public static<T> Stream<T> of(T... values) {  
        return Arrays.stream(values);  
    }  
}
```



```
public class java.util.Arrays {  
    public static <T> Stream<T> stream(T[] array) {  
        return stream(array, 0, array.length);  
    }  
}
```



What is the difference between these two calls?

## Stream.of([]) vs. Arrays.stream([])

```
public interface Stream<T> extends BaseStream<T, Stream<T>> {  
    public static<T> Stream<T> of(T... values) {  
        return Arrays.stream(values);  
    }  
}
```



```
public class java.util.Arrays {  
    public static <T> Stream<T> stream(T[] array) {  
        return stream(array, 0, array.length);  
    }  
}
```



They are “almost” equivalent, however favor **Arrays.stream([])** to avoid an indirection call and *varargs* parameters 😊

# Parallel Streams

```
@Param({ "8192", "8193" })
int arraySize;

List<String> list = new ArrayList<>(arraySize);

@Benchmark
public Object[] sort() {
    Object[] result = list.parallelStream()
        .sorted()
        .toArray();

    return result;
}
```

```

@Param({ "8192", "8193" })
int arraySize;

List<String> list = new ArrayList<>(arraySize);

@Benchmark
public Object[] sort() {
    Object[] result = list.parallelStream()
        .sorted()
        .toArray();

    return result;
}

```

| arraySize | us/op |
|-----------|-------|
| 8 192     | 1 712 |
| 8 193     | 945   |

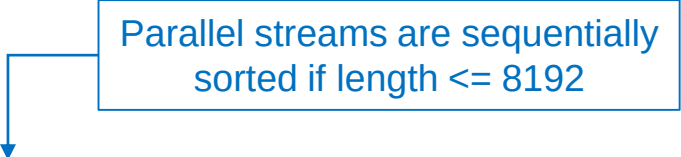
?

Why is 8 193 arraySize ~2x faster?



# Solution

```
public class java.util.Arrays {  
  
    public static <T> void parallelSort(T[] a, Comparator<? super T> cmp) {  
  
        if (cmp == null)  
            cmp = NaturalOrder.INSTANCE;  
        int n = a.length, p, g;  
  
        if ( n <= MIN_ARRAY_SORT_GRAN ) // n <= 8192  
            TimSort.sort(...);  
        else  
            new ArraysParallelSortHelpers.FJObject.Sorter<>(...).invoke();  
  
    }  
}
```



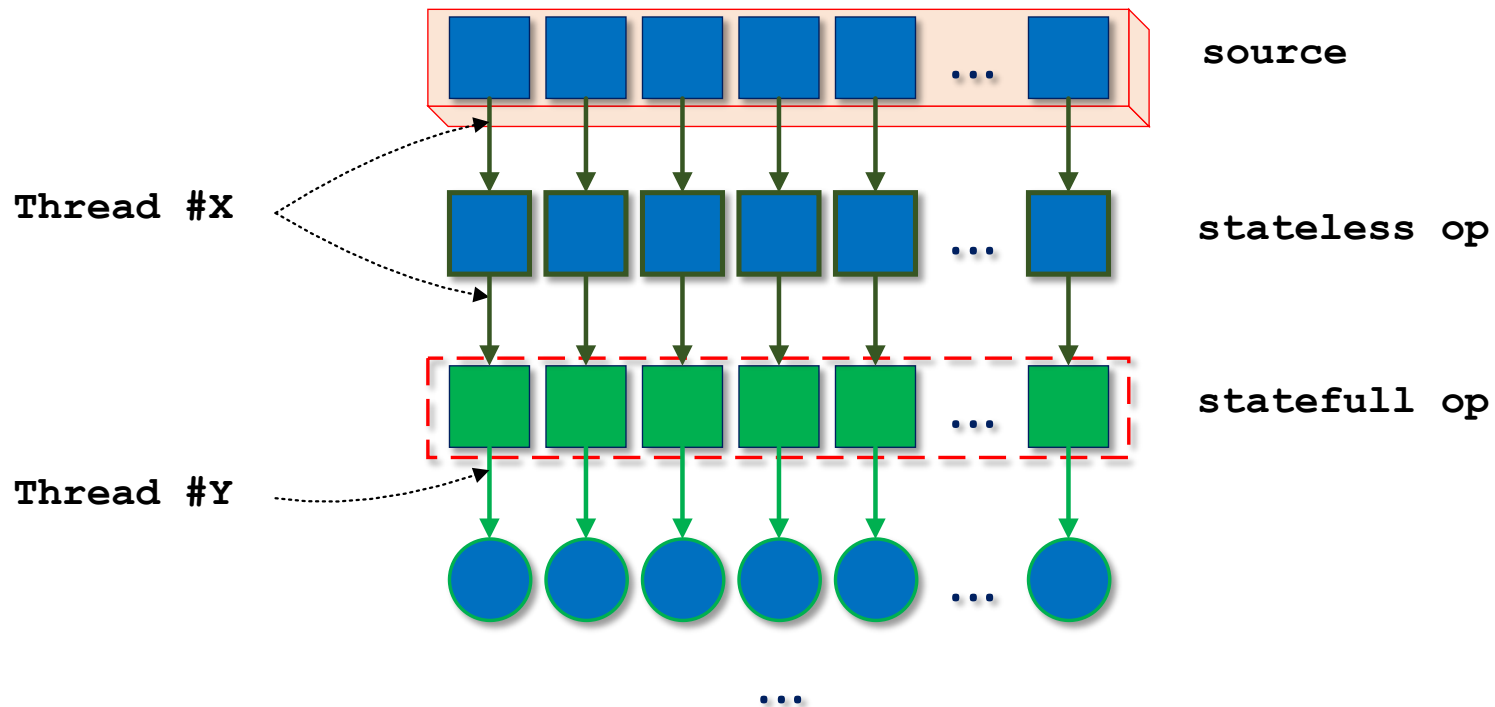
Parallel streams are sequentially sorted if length <= 8192

**NQ model** - larger the product  $N \times Q$  is more likely to get a parallel speedup !

$N$  - number of data elements;  $Q$  - amount of work performed per element

For problems with trivially small  $Q$  (e.g. sort, add) generally  $N > 10,000$  to get a speedup !

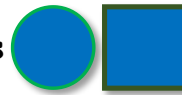
# ForkJoinPool



FJ Threads



FJ Tasks





## How many default threads are used by the ForkJoinPool?

```
Set<String> threads = new TreeSet<>();  
Object[] result = list.parallelStream()  
    .filter(s -> {  
        threads.add(Thread.currentThread().getName());  
        return true;  
    })  
    .sorted()  
    .toArray();
```



## How many default threads are used by the ForkJoinPool?

```
Set<String> threads = new TreeSet<>();  
Object[] result = list.parallelStream()  
    .filter(s -> {  
        threads.add(Thread.currentThread().getName());  
        return true;  
    })  
    .sorted()  
    .toArray();
```

8 threads in total

```
ForkJoinPool.commonPool-worker-11  
ForkJoinPool.commonPool-worker-13  
ForkJoinPool.commonPool-worker-2  
ForkJoinPool.commonPool-worker-4  
ForkJoinPool.commonPool-worker-6  
ForkJoinPool.commonPool-worker-8  
ForkJoinPool.commonPool-worker-9  
main
```

```
ForkJoinPool.commonPool-worker-11  
ForkJoinPool.commonPool-worker-13  
ForkJoinPool.commonPool-worker-2  
ForkJoinPool.commonPool-worker-4  
ForkJoinPool.commonPool-worker-6  
ForkJoinPool.commonPool-worker-8  
ForkJoinPool.commonPool-worker-9  
main
```

e.g. # of Threads ← `Runtime.getRuntime().availableProcessors() - 1 + main` thread



Parallel stream is synchronous for the invoking thread (i.e. `main` thread)

```
ForkJoinPool.commonPool-worker-11  
ForkJoinPool.commonPool-worker-13  
ForkJoinPool.commonPool-worker-2  
ForkJoinPool.commonPool-worker-4  
ForkJoinPool.commonPool-worker-6  
ForkJoinPool.commonPool-worker-8  
ForkJoinPool.commonPool-worker-9  
main
```

e.g. # of Threads ← `Runtime.getRuntime().availableProcessors() - 1 + main` thread



Parallel stream is synchronous for the invoking thread (i.e. `main` thread)

To explicitly set number of threads use System property  
`-Djava.util.concurrent.ForkJoinPool.common.parallelism=<threads>`



All parallel streams share the same ForkJoinPool inside JVM!

Enforcing parallel stream to use a custom dedicated ForkJoinPool:

```
List<String> list = new ArrayList<>(10_000);

ForkJoinPool fjp = new ForkJoinPool(4);
ForkJoinTask<Object[]> task = fjp.submit(() -> {
    return list.parallelStream()
        .sorted()
        .toArray();
});

Object[] result = task.get();
```



# ForkJoinPool Algorithm

**Result** execute(**Problem** problem)

```
{  
    if (problem too small)  
        directly solve problem  
    else  
    {  
        split problem into independent parts  
        fork new subtasks to solve each part  
        join all subtasks  
        compose result from sub-results  
    }  
}
```

# ForkJoinPool Algorithm - Example

Result execute(**Problem** problem)

{

if (**problem too small**)

directly solve problem

else

$\text{streamEstimatedSize} / [\text{ForkJoinPool.getCommonPoolParallelism()} \ll 2]$

{

e.g.

$\text{streamEstimatedSize} \leftarrow 10\_000$

$\text{ForkJoinPool.getCommonPoolParallelism()} \ll 2 = 8 \ll 2 = 32$

**problem too small**  $\Leftrightarrow$  size is less than 312 (i.e.  $10\_000 / 32$ )

compose result from sub-results

}

*-Djava.util.concurrent.ForkJoinPool.common.parallelism = 8*

# String in Java 9

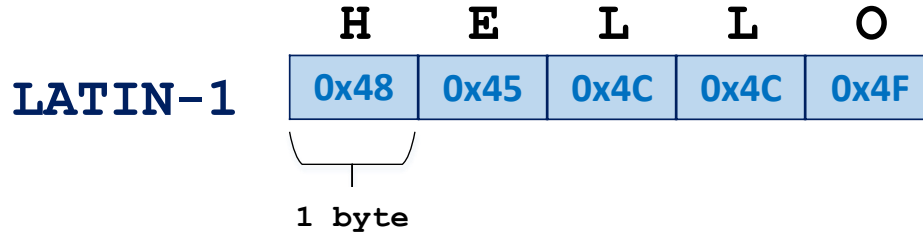
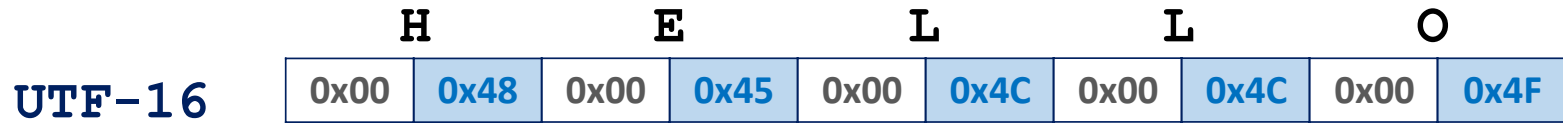
# JEP 254: Compact Strings

```
public final class String {  
  
    private final byte[] value;  
    private final byte coder;  
    private int hash;  
  
}
```

```
coder == LATIN1 || UTF16
```

```
static final byte LATIN-1 = 0;  
static final byte UTF-16  = 1;
```

**-XX:+CompactStrings** - enabled by default



***CODER** field indicates which encoding is used*

Strings containing characters with non-zero upper byte cannot be compressed!

- stored as 2 byte characters using **UTF-16** encoding

Strings containing only characters with zero upper byte can be compressed to **Latin-1**!

- high-order zero bytes are stripped of

# Challenges

## String construction

- allocate a `byte[]` and tries to compress the input `char[]`
- if it fails, uses the `char[]`

## Inflation: `byte[]` → `char[]`

- whenever we need a `char[]` representation (e.g. `toCharArray()`)



`compare()` does not really touch real values if the **coder** differs!

```
@Param({"42"})  
public int x;
```

```
@Param({"BJUG 2018"})  
public String msg;
```

```
@Param({"[42] This is a private welcome message: BJUG 2018"})  
public String utf_8;
```

```
@Param({"[42] ĐžĐ¹, Đ²ÑÑ\ Đ¿Ñ€Đ³Đ¿Đ°Đ»Đ¼, Ñ^ĐµÑ,, : BJUG 2018"})  
public String utf_16;
```

```
@Benchmark
public String utf8() {
    return "[" + x + "] This is a private welcome message: " + msg;
}
```

```
@Benchmark
public String utf16() {
    return "[" + x + "] ĐŽĐ¹, Đ²ÑÑ\ Đ¿Ñ€Đ¼Đ¿Đ°Đ»Đ¼, Ñ^ĐµÑ„: " + msg;
}
```

```
@Benchmark
public char[] utf8_toCharArray() {
    return utf_8.toCharArray();
}
```

```
@Benchmark
public char[] utf16_toCharArray() {
    return utf_16.toCharArray();
}
```



|                   | JDK 8 |          | JDK 9 |          |
|-------------------|-------|----------|-------|----------|
|                   | ns/op | GC count | ns/op | GC count |
| utf8              | 20,9  | 269      | 20,3  | 185      |
| utf16             | 20,8  | 267      | 22,5  | 257      |
| utf8_toCharArray  | 10,8  |          | 10,9  |          |
| utf16_toCharArray | 10,8  |          | 11,7  |          |

utf8 / utf16 String creation:

- **JDK 8** - almost same op performance and # of GC cycles
- **JDK 9** - better op performance and less # of GC cycles for utf8

toCharArray() - in case of utf16 the cost of inflation from byte[] to char[] is noticeable

```
public String utf8() {  
  
    String x = "42";  
    String msg = "BJUG 2018";  
    return "[" + x + "] This is a private welcome message: " + msg;  
  
}
```

?

How will this be compiled at Bytecode level (JDK 8 / JDK 9)?

# JDK 8

```
public utf8() LString;
```

```
NEW java/lang/StringBuilder
```

```
DUP
```

```
INVOKESPECIAL java/lang/StringBuilder.<init> ()V
```

```
LDC "["
```

```
INVOKEVIRTUAL java/lang/StringBuilder.append (LString;) LStringBuilder;
```

```
LDC "42"
```

```
INVOKEVIRTUAL java/lang/StringBuilder.append (LString;) LStringBuilder;
```

```
LDC "]" This is a private welcome message: "
```

```
INVOKEVIRTUAL java/lang/StringBuilder.append (LString;) LStringBuilder;
```

```
LDC "BJUG 2018"
```

```
INVOKEVIRTUAL java/lang/StringBuilder.append (LString;) LStringBuilder;
```

```
INVOKEVIRTUAL java/lang/StringBuilder.toString () LString;
```

# JDK 8

```
public utf8() LString;
```

```
    NEW java/lang/StringBuilder
```

```
    DUP
```

```
    INVOKESPECIAL java/lang/StringBuilder.<init> ()V
```

new StringBuilder()

```
    LDC "["
```

```
    INVOKEVIRTUAL java/lang/StringBuilder.append (LString;) LStringBuilder;
```

```
    LDC "42"
```

```
    INVOKEVIRTUAL java/lang/StringBuilder.append (LString;) LStringBuilder;
```

```
    LDC "]" This is a private welcome message: "
```

```
    INVOKEVIRTUAL java/lang/StringBuilder.append (LString;) LStringBuilder;
```

```
    LDC "BJUG 2018"
```

```
    INVOKEVIRTUAL java/lang/StringBuilder.append (LString;) LStringBuilder;
```

```
    INVOKEVIRTUAL java/lang/StringBuilder.toString () LString;
```

Strings literals concatenation

# JDK 9

```
public utf8() LString;
```

```
LDC "42"
```

```
LDC "BJUG 2018"
```

```
INVOKEDYNAMIC (
```

```
  ( MH(StringConcatFactory),
```

```
    MH(INVOKESTATIC StringConcatFactory.makeConcatWithConstants),
```

```
    // arguments:
```

```
    "[\u0001] This is a private welcome message: \u0001"
```

```
    ( )
```

```
  )
```

```
)
```

# JDK 9

## JEP 280: Indify String Concatenation

```
public utf8() LString;
```

```
LDC "42"  
LDC "BJUG 2018"  
INVOKEDYNAMIC (  
    ( MH(StringConcatFactory),  
      MH(INVOKESTATIC StringConcatFactory.makeConcatWithConstants),  
      // arguments:  
      "[\u0001] This is a private welcome message: \u0001"  
      ( )  
    )  
)
```

**Motivation:** built in optimized String concatenation handlers, implementable without need to change the Java-to-bytecode compiler.

**BREAK**

# Stack Walker



Prior JDK 9 APIs require VM to eagerly capture a entire stack snapshot, hence they return information representing the full stack (i.e. performance penalty).

## JDK $\leq$ 8 API

- `Throwable::getStackTrace`
- `Thread::getStackTrace` - may return a partial stack (i.e. performance considerations <sup>[\*]</sup>)
- `SecurityManager::getClassContext` - allows accessing actual instance of the class that declared the method represented by the stack frame

<sup>[\*]</sup> `-XX:+OmitStackTraceInFastThrow` - when an exception is re-thrown few times the method may be re-compiled and compiler may choose a faster tactic using pre-allocated exceptions omitting a stack trace

# JEP 259: Stack-Walking API

**Stack-Walker** API is a flexible mechanism to traverse and materialize the required stack-frame information allowing efficient lazy access to additional stack frames when required.

It avoids the cost of examining all frames if caller is only interested in a few.

## **Stack-Walker features:**

- limit stack depth
- filter frames
- lazy frames construction

StackWalker **getInstance**()

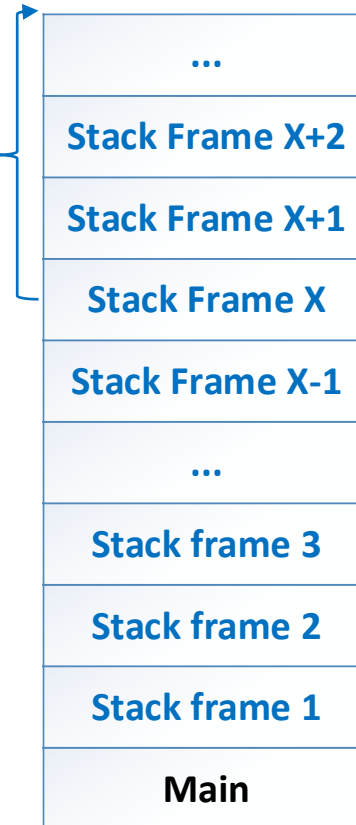
StackWalker **getInstance**(Set<Option> options)

**Option:**

```
// support for getCallerClass() && getDeclaringClass()  
RETAIN_CLASS_REFERENCE  
// support reflexion frames for  
// reflect.Method#invoke && reflect.Constructor#newInstance  
SHOW_REFLECT_FRAMES  
// support for all hidden frames  
SHOW_HIDDEN_FRAMES
```

StackWalker **getInstance**(Set<Option> options, **int** estimateDepth)

retrieve last N stack frames  
- *stack traversed backwards N frames* -



# Retrieve Last N Stack Frames - Stack Walker

```
@Benchmark
public StackWalker.StackFrame retrieveLastNStackFrames () {
    return recNStackFrames (stackDepth);
}

private StackWalker.StackFrame recNStackFrames (int depth) {
    if (depth == 0) {
        return getNStackFrames (stackDepth);
    }
    return recNStackFrames (depth - 1);
}

private StackWalker.StackFrame getNStackFrames (int numberOfSkippedFrames) {
    return StackWalker.getInstance()
        .walk(stream -> stream.skip(numberOfSkippedFrames - 1).findFirst())
        .orElseThrow (NoSuchElementException::new);
}
```

# Retrieve Last N Stack Frames - `getStackTrace()`

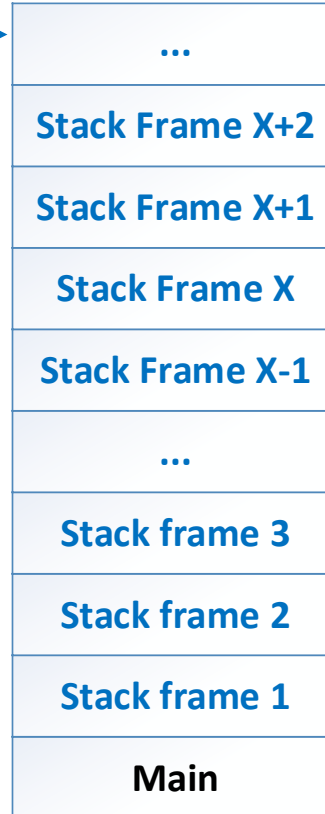
```
private StackTraceElement getNStackFrames(int numberOfSkippedFrames) {  
    StackTraceElement[] stackTrace = Thread.currentThread().getStackTrace();  
    return Arrays.stream(stackTrace).skip(numberOfSkippedFrames - 1).findFirst()  
        .orElseThrow(NoSuchElementException::new);  
}
```

# Retrieve Last N Stack Frames

| StackDepth      | 1     | 10    | 100   | 1000  |
|-----------------|-------|-------|-------|-------|
|                 | us/op | us/op | us/op | us/op |
| StackWalker     | 1,7   | 5,5   | 33,3  | 308,0 |
| getStackTrace() | 13,4  | 17,9  | 61,0  | 482,6 |

**StackWalker** offers better performance for traversing few stack frames, however it becomes costly for bigger StackDepth

retrieve current stack frame  
- *current top frame* -





# Retrieve Current Stack Frame - Stack Walker

```
@Benchmark
public StackWalker.StackFrame retrieveLastStackFrame () {
    return recLastStackFrame(stackDepth);
}

private StackWalker.StackFrame recLastStackFrame(int depth) {
    if (depth == 0) {
        return getLastStackFrame();
    }
    return recLastStackFrame(depth - 1);
}

private StackWalker.StackFrame getLastStackFrame() {
    return StackWalker.getInstance()
        .walk(stream -> stream.findFirst())
        .orElseThrow(NoSuchElementException::new);
}
```

# Retrieve Current Stack Frame - `getStackTrace()`

```
private StackTraceElement getLastStackFrame() {  
    StackTraceElement[] stackTrace = Thread.currentThread().getStackTrace();  
    return Arrays.stream(stackTrace).findFirst()  
        .orElseThrow(NoSuchElementException::new);  
}
```

# Retrieve Current Stack Frame

| StackDepth      | 1     | 10    | 100   | 1000  |
|-----------------|-------|-------|-------|-------|
|                 | us/op | us/op | us/op | us/op |
| StackWalker     | 1,7   | 1,7   | 1,8   | 3,4   |
| getStackTrace() | 13,2  | 17,6  | 61,0  | 474,6 |

**StackWalker** offers 1 order of magnitude performance and almost constant access to current top frame no matter how big StackDepth is

**getStackTrace()** is proportional to StackDepth

# Collection Factory Methods

# JEP 269: Convenience Factory Methods for Collections

**Factory Methods** - static factory methods on the collection interfaces that will create compact, unmodifiable collection instances.

```
List.of()  
List.of(e1)  
List.of(e1, e1)  
List.of(e1, e1, e3)  
...
```

```
Set.of()  
Set.of(e1)  
Set.of(e1, e2)  
Set.of(e1, e2, e3)  
...
```

```
Map.of()  
Map.of(k1, v1)  
Map.of(k1, v1, k2, v2)  
Map.of(k1, v1, k2, v2, k3, v3)  
...
```

# List.of()

# List Wrappers

```
static <E> List<E> of() {  
    return ImmutableList.of(List0.instance());  
}
```

```
static <E> List<E> of(E e1) {  
    return new ImmutableList.List1<>(e1);  
}
```

```
static <E> List<E> of(E e1, E e2) {  
    return new ImmutableList.List2<>(e1, e2);  
}
```

```
static <E> List<E> of(E e1, E e2, E e3) {  
    return new ImmutableList.ListN<>(e1, e2, e3);  
}
```

# List Wrappers

```
static <E> List<E> of() {  
    return ImmutableCollections.List0.instance();  
}  
  
static <E> List<E> of(E e1) {  
    return new ImmutableCollections.List1<>(e1);  
}  
  
static <E> List<E> of(E e1, E e2) {  
    return new ImmutableCollections.List2<>(e1, e2);  
}  
  
static <E> List<E> of(E e1, E e2, E e3) {  
    return new ImmutableCollections.ListN<>(e1, e2, e3);  
}
```

specific wrappers for 0...2 elements

generic wrapper for  $N > 2$  elements



```
static final class List0<E> extends AbstractImmutableList<E> {  
    private static final List0<?> INSTANCE = new List0<>();  
    //...  
}
```

```
static final class List1<E> extends AbstractImmutableList<E> {  
    private final E e0;  
    //...  
}
```

```
static final class List2<E> extends AbstractImmutableList<E> {  
    private final E e0;  
    private final E e1;  
    //...  
}
```

```
static final class ListN<E> extends AbstractImmutableList<E> {  
    private final E[] elements;  
    //...  
}
```

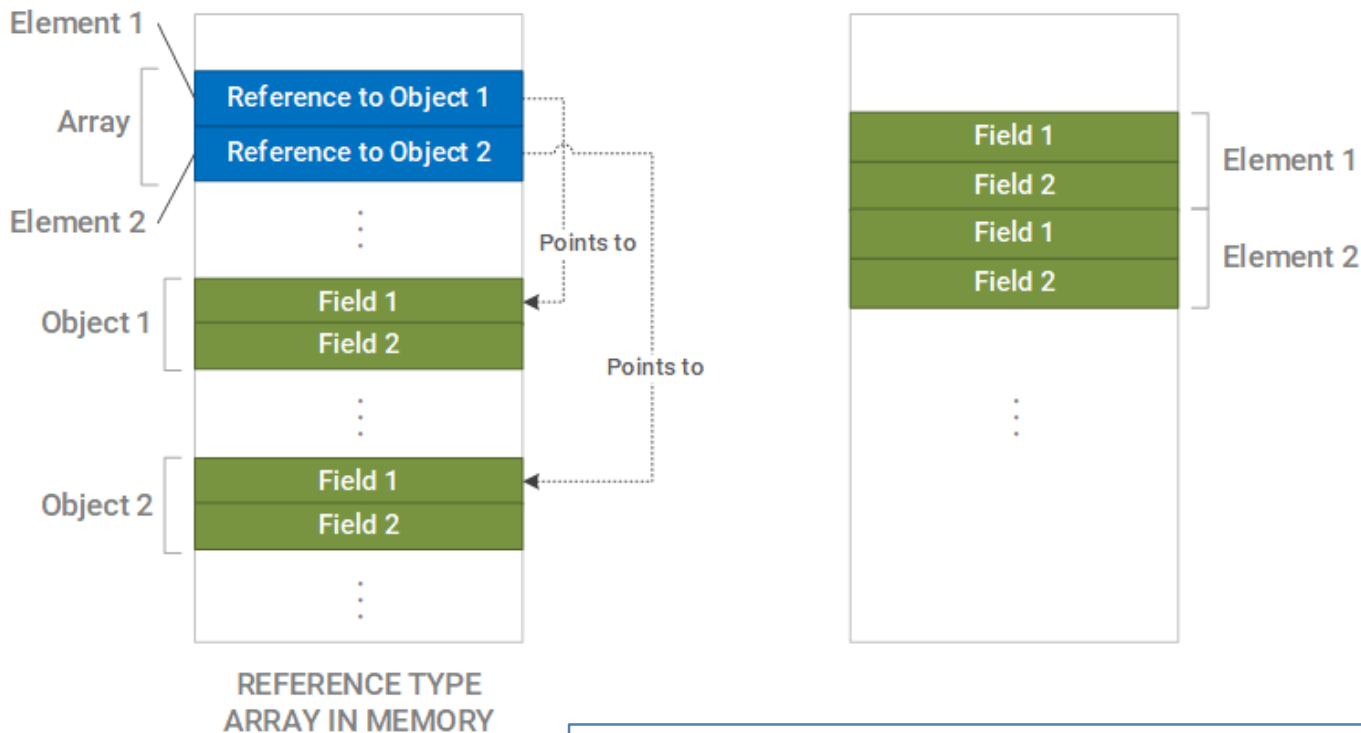
?

Why do you think there are explicit wrappers for **List0**, **List1** and **List2**?

?

Why do you think there are explicit wrappers for **List0**, **List1** and **List2**?

DATA LOCALITY



?

Why are there **of()** methods explicitly declared for first 10 params?

```
<E> List<E> of()
<E> List<E> of(E e1)
<E> List<E> of(E e1, E e2)
<E> List<E> of(E e1, E e2, E e3)
<E> List<E> of(E e1, E e2, E e3, E e4)
<E> List<E> of(E e1, E e2, E e3, E e4, E e5)
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6)
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7)
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8)
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9)
<E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9, E e10)

<E> List<E> of(E... elements) // varargs
```



Why are there **of()** methods explicitly declared for first 10 params?

**var\_args\_11\_params** method call

```
prefetchw BYTE PTR [r10+0x180] ;*newarray
```

```
mov    r10d,DWORD PTR [rbp+0x10] ;*getfield param2
mov    r8d,DWORD PTR [rbp+0x14] ;*getfield param3
mov    r11d,DWORD PTR [rbp+0x18] ;*getfield param4
mov    ecx,DWORD PTR [rbp+0x1c] ;*getfield param5
mov    r9d,DWORD PTR [rbp+0x20] ;*getfield param6
mov    edi,DWORD PTR [rbp+0x24] ;*getfield param7
mov    ebx,DWORD PTR [rbp+0x28] ;*getfield param8
mov    esi,DWORD PTR [rbp+0x2c] ;*getfield param9
mov    edx,DWORD PTR [rbp+0x30] ;*getfield param10
mov    edx,DWORD PTR [rbp+0x34] ;*getfield param11
mov    eax,DWORD PTR [rbp+0xc] ;*getfield param1
```



**varargs** method call allocates a new array! (i.e. performance impact)

| ns/op         |      |                    |
|---------------|------|--------------------|
| # of elements | of() | unmodifiableList() |
| empty         | 2,6  | 7,6                |
| 1             | 4,8  | 16,8               |
| 2             | 5,9  | 20,7               |
| 3             | 12,5 | 23,8               |
| 10            | 25,9 | 48,0               |
| 11            | 27,4 | 61,5               |

[List.of\(\)](#) offers better performance in comparison to [Collections.unmodifiableList\(\)](#)

# Set.of()

# Set Wrappers

```
static final class Set0<E> extends AbstractImmutableSet<E> {...}
```

```
static final class Set1<E> extends AbstractImmutableSet<E> {...}
```

```
static final class Set2<E> extends AbstractImmutableSet<E> {...}
```

```
static final class SetN<E> extends AbstractImmutableSet<E> {...}
```



# Set.of()

```
<E> Set<E> of()
<E> Set<E> of(E e1)
<E> Set<E> of(E e1, E e2)
<E> Set<E> of(E e1, E e2, E e3)
<E> Set<E> of(E e1, E e2, E e3, E e4)
<E> Set<E> of(E e1, E e2, E e3, E e4, E e5)
<E> Set<E> of(E e1, E e2, E e3, E e4, E e5, E e6)
<E> Set<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7)
<E> Set<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8)
<E> Set<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9)
<E> Set<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9, E e10)

<E> Set<E> of(E... elements) // varargs
```

| ns/op         |       |                   |
|---------------|-------|-------------------|
| # of elements | of()  | unmodifiableSet() |
| empty         | 2,6   | 10,4              |
| 1             | 4,0   | 25,6              |
| 2             | 8,0   | 40,5              |
| 3             | 42,5  | 51,6              |
| 10            | 100,9 | 128,3             |
| 11            | 110,9 | 139,5             |

**Set.of()** offers better performance in comparison to **Collections.unmodifiableSet()**

# Map.of()

# Map Wrappers

```
static final class Map0<K, V> extends AbstractImmutableMap<K, V> {...}
```

```
static final class Map1<K, V> extends AbstractImmutableMap<K, V> {...}
```

```
static final class MapN<K, V> extends AbstractImmutableMap<K, V> {...}
```

# Map.of()

```
<K, V> Map<K, V> of()
<K, V> Map<K, V> of(K k1, V v1)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4, K k5, V v5)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4, K k5, V v5,
                    K k6, V v6)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4, K k5, V v5,
                    K k6, V v6, K k7, V v7)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4, K k5, V v5,
                    K k6, V v6, K k7, V v7, K k8, V v8)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4, K k5, V v5,
                    K k6, V v6, K k7, V v7, K k8, V v8, K k9, V v9)
<K, V> Map<K, V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4, K k5, V v5,
                    K k6, V v6, K k7, V v7, K k8, V v8, K k9, V v9, K k10, V v10)

<K, V> Map<K, V> ofEntries(Entry<? extends K, ? extends V>... entries) // varargs
```

| ns/op         |       |             |                   |
|---------------|-------|-------------|-------------------|
| # of elements | of()  | ofEntries() | unmodifiableMap() |
| empty         | 2,5   | 2,5         | 8,6               |
| 1             | 5,9   | 9,3         | 23,4              |
| 2             | 38,1  | 55,5        | 38,1              |
| 3             | 48,0  | 75,8        | 49,5              |
| 4             | 65,6  |             | 61,4              |
| 10            | 156,9 | 248,4       | 128,5             |
| 11            |       | 263,0       | 139,4             |

**Map.of()** offers better performance in comparison to **Map.ofEntries()**

**Collections.unmodifiableMap()** offers quite equivalent performance as **Map.of()**

# Contended locks in Java 9

# JEP 143: Improve Contended Locking

Improve the overall performance of contended Java object monitors.

**Contended Lock** - a thread wants to enter the synchronized method that is already locked by another thread

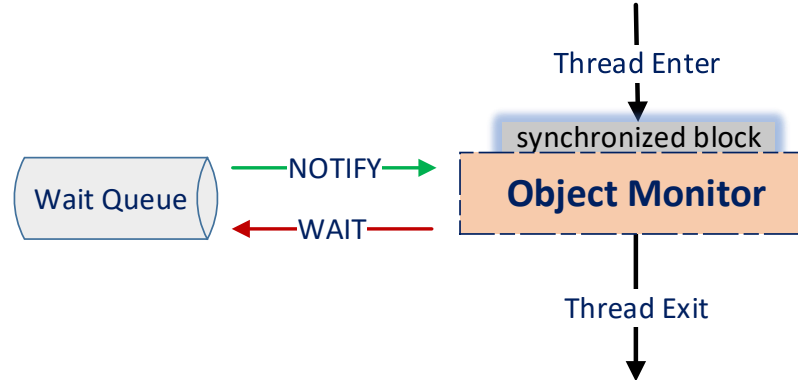
## Few related areas:

- Fast Java monitor enter operations
- Fast Java monitor exit operations
- Fast Java monitor **notify/notifyAll** operations

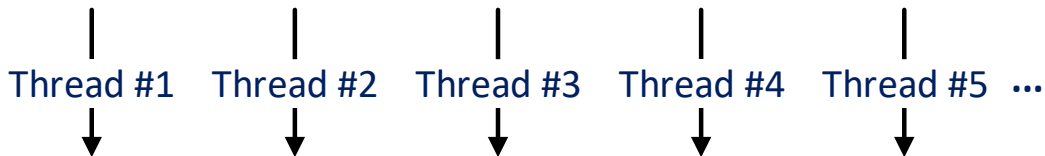


Every Object has an associated:

- monitor
- “WaitSet” for waiting threads (i.e. heavy weight locking scheme)



```
final long ITERATIONS = 10 * 1000 * 1000;  
final Object lock = new Object();  
volatile long consumeCPU = 0;
```



```
long start = System.nanoTime();  
  
for (int i = 0; i < ITERATIONS; i++) {  
    synchronized (lock) { // high contention on the shared lock  
        consumeCPU += ++consumeCPU & 0xF;  
    }  
}  
  
long stop = System.nanoTime();  
hist.recordValue(stop-start);
```

Default in JDK 9

| # of threads | sec/op                                         |                                                |
|--------------|------------------------------------------------|------------------------------------------------|
|              | -XX:-UseBiasedLocking<br>-XX:+UseHeavyMonitors | -XX:+UseBiasedLocking<br>-XX:-UseHeavyMonitors |
| 40           | 2,1                                            | 2,0                                            |
| 80           | 2,5                                            | 2,8                                            |
| 160          | 4,2                                            | 5,3                                            |
| 240          | 6,3                                            | 7,2                                            |
| 320          | 8,6                                            | 9,9                                            |

Contended locks offers better performance when the number of contended threads is significant higher!

# Bibliography

Streams under the hood - Brian Goetz

- <https://www.ibm.com/developerworks/library/j-java-streams-3-brian-goetz/index.html>

State of the Lambda - Brian Goetz

- <http://cr.openjdk.java.net/~briangoetz/lambda/lambda-state-final.html>

Translation of Lambda Expressions - Brian Goetz

- <http://cr.openjdk.java.net/~briangoetz/lambda/lambda-translation.html>

From Concurrent to Parallel - Brian Goetz

- <https://www.youtube.com/watch?v=tjnGtDbU8l8>

Compact Strings - Aleksey Shipilëv

- <https://www.youtube.com/watch?v=wlyeOaitmWM>

Java 9: Beyond Contention! - Monica Beckwith

- [https://www.youtube.com/watch?v=q1CeN\\_mR-II](https://www.youtube.com/watch?v=q1CeN_mR-II)

Everything you wanted to know about Stack Traces - Andrei Pangin

- <https://www.youtube.com/watch?v=FTsAXJdrJbl>

**Find out more about our Software  
Architecture and Java courses**

**WWW.LUXOFT-TRAINING.RO**

# APPENDIX

# Predicate<T> Bytecode

```
public static public static predicate_int()Z
    INVOKEDYNAMIC (
        ( MH(metaFactory),
          MH(INVOKEINTERFACE java/util/function/Predicate.test),
          MH(INVOKESTATIC PredicateJmh.lambda$predicate_int$0(LInteger;)Z)
            ( intValue )
        )
    )
    INVOKEINTERFACE java/util/function/Predicate.test (LObject;)Z

private static synthetic lambda$predicate_int$0(LInteger;)Z
    ALOAD 0
    INVOKEVIRTUAL java/lang/Integer.intValue ()I
    ICONST_2
    ISHL
    IFLE L1
    ICONST_1
    IRETURN
L1
    ICONST_0
```

Desugared the same way as lambdas + interface method calls

**Do you think of some possible  
issues?**



**Do you think of some possible  
issues?**

<< Hmm ... may be inlining!? >>

## -XX:+PrintInlining

depth3

```
@ 30    LambdaMain$$Lambda$51/731395981::next (8 bytes)      inline (hot)
@ 35    LambdaMain$$Lambda$50/13326370::next (8 bytes)      inline (hot)
@ 40    LambdaMain$$Lambda$49/146611050::next (8 bytes)      inline (hot)
@ 45    LambdaMain$$Lambda$48/214074868::get (4 bytes)       inline (hot)
    \-> TypeProfile (10666/10666 counts) = LambdaMain$$Lambda$48
        @ 0    LambdaMain::lambda$depth3$5 (6 bytes)        inline (hot)
            @ 2    java.lang.Integer::valueOf (32 bytes)    inline (hot)
@ 50    java.lang.Integer::intValue (5 bytes)    accessor
@ 4     LambdaMain::<init> (11 bytes)
        @ 1     java.lang.Object::<init> (1 bytes)
@ 22    com.jpt.lambda.LambdaMain::depth3 (54 bytes)    callee is too large
```

?

Does increasing inlining threshold really help?

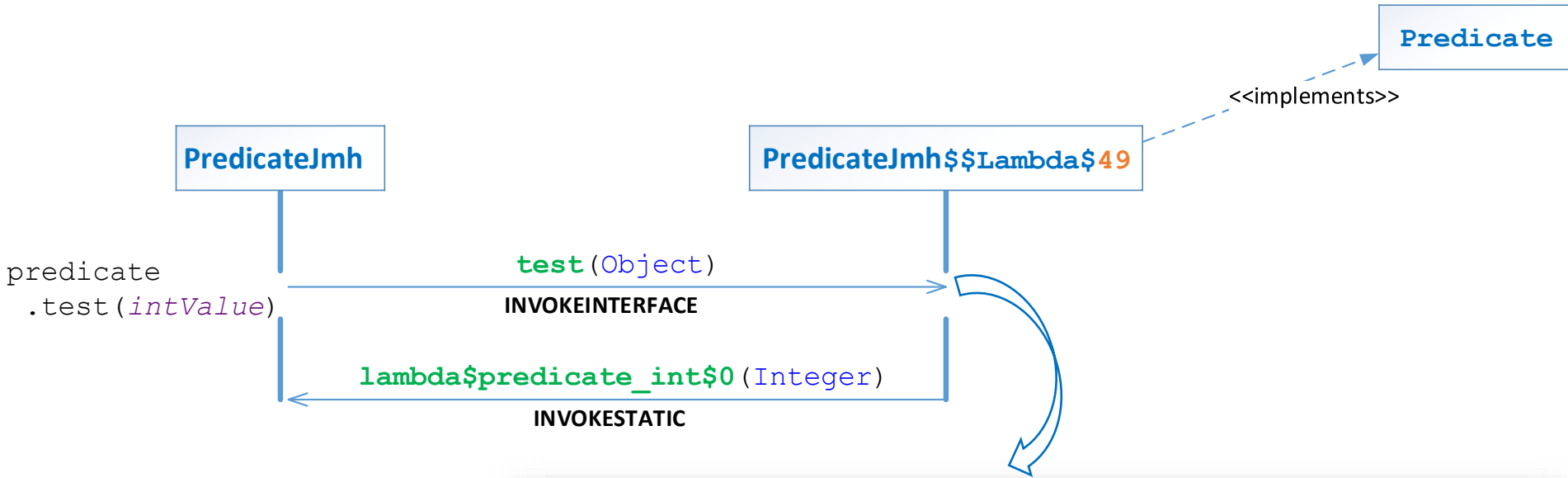
**-XX:+PrintInlining** **-XX:MaxInlineSize=54**

**depth3**

```
@ 30    LambdaMain$$Lambda$51/731395981::next (8 bytes)
@ 35    LambdaMain$$Lambda$50/13326370::next (8 bytes)
@ 40    LambdaMain$$Lambda$49/146611050::next (8 bytes)
@ 45    LambdaMain$$Lambda$48/214074868::get (4 bytes)
    @ 0    LambdaMain::lambda$depth3$5 (6 bytes)
        @ 2    java.lang.Integer::valueOf (32 bytes)
```

Not really, since it refers to inline **depth3** in the caller (i.e. **main**)

# Predicate<T>



```
public boolean test(Object var1) {  
    return  
        PredicateJmh.lambda$predicate_int$0((Integer) var1);  
}
```

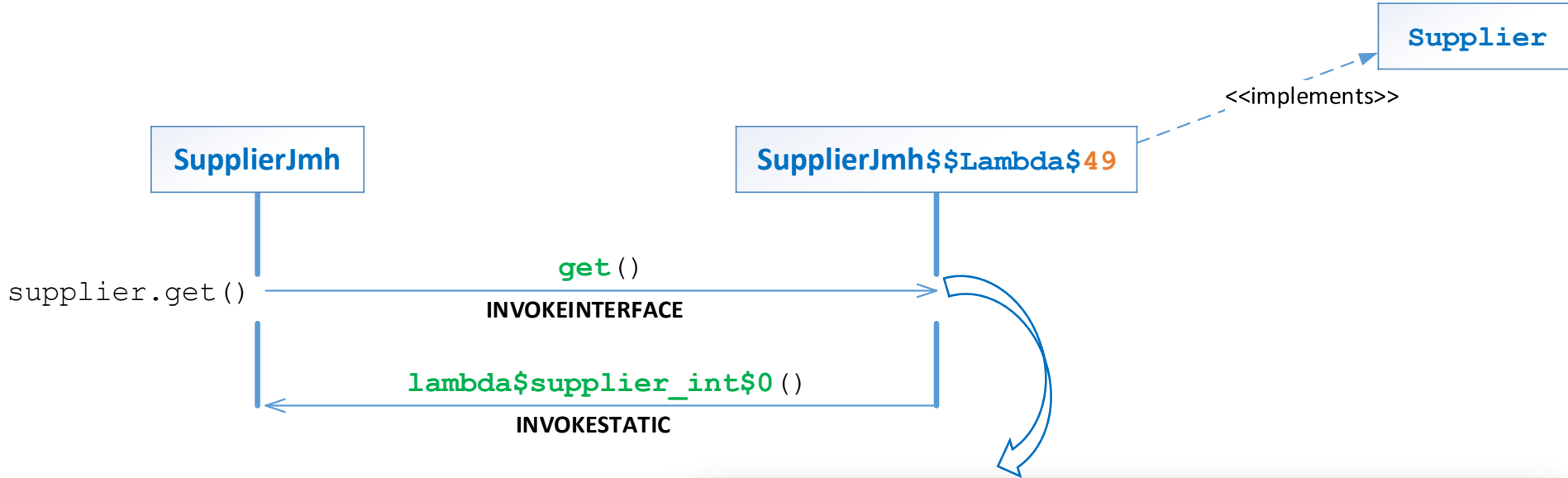
# Supplier<T> Bytecode

```
public static supplier_int()Ljava/lang/Integer;
    INVOKEDYNAMIC (
        ( MH(metaFactory),
          MH(INVOKEINTERFACE java/util/function/Supplier.get),
          MH(INVOKESTATIC SupplierJmh.lambda$supplier_int$0()Ljava/lang/Integer)
            ( )
          )
    )
    INVOKEINTERFACE java/util/function/Supplier.get ()Ljava/lang/Object;

private static synthetic lambda$supplier_int$0()Ljava/lang/Integer;
    GETSTATIC com/jpt/lambda/SupplierJmh.intValue : Ljava/lang/Integer;
```

Desugared the same way as lambdas + interface method calls

# Supplier<T>



```
public Object get() {  
    return SupplierJmh.lambda$supplier_int$0();  
}
```

# Stream Flags - Examples

```
public interface java.util.Spliterator<T> {  
  
    /** size of the stream is known */  
    public static final int SIZED          = 0x00000040;  
  
    /** elements of the stream are distinct */  
    public static final int DISTINCT      = 0x00000001;  
  
    /** elements of the stream are sorted in the natural order */  
    public static final int SORTED        = 0x00000004;  
  
    /** stream has a meaningful encounter order */  
    public static final int ORDERED       = 0x00000010;  
  
}
```

# Stream Flags and Performance Impact



**sort()** - if **ORDER** is significant, sorted operation implements a *stable sort* (i.e. equal elements appear in the same order in the output as in the input), whereas for an unordered stream isn't required



**distinct()** - if **ORDER** is significant, then for multiple equal input elements, distinct must emit the first of them, whereas for an unordered stream, it can emit any of them



# Other Examples - Redundant Operations

```
HashSet<String> hashSet = new HashSet<>();
```

```
@Benchmark
```

```
public Object[] hashSet_explicit_distinct() {
```

```
    Object[] result = hashSet.parallelStream() ←
```

```
        .filter(s -> s.startsWith("i"))
```

```
        .distinct() ←
```

```
        .toArray();
```

```
    return result;
```

```
}
```

implicitly distinct

✓ no-op (skips distinct)

| HashSet – parallel stream | us/op |
|---------------------------|-------|
| implicit distinct         | 71    |
| explicit distinct         | 73    |

Explicit distinct still have a bit of penalty for no-op in comparison to implicit sort

# Map.ofEntry()

starting to the 11<sup>th</sup> <K, V> pair, API for creating an immutable Map is

```
Map.ofEntries(  
    entry(k1, v1),  
    entry(k2, v2),  
    entry(k3, v3),  
    // ...  
    entry(kn, vn)  
);
```

which unfortunately requires each <K, V> pair be boxed!

# A Bit of History ...

**UseCompressedStrings** - introduced by Java 6 u 21

- an optional and experimental, non open source feature
- tried to compress **char[]** into **byte[]** on String construction, hence it did most String operations on char[], which required a lot of unpacking from byte[] to char[] since most of Strings could fit on byte (e.g. LATIN-1)
- feature implemented in **alt-rt.jar**. Optional features are harder to test, since they double the number of option combinations to try.

Source [<https://www.infoq.com/news/2016/02/compact-strings-Java-JDK9>]

# A Bit of History ...

**UseCompressedStrings** - introduced by Java 6 u 21

➤ an optional and experimental, non open source feature

➤  Let's reuse the same idea however instead of switching between `byte[]` ↔ `char[]` to have String directly backed by `byte[]` whenever possible

➤ feature implemented in **alt-rt.jar**. Optional features are harder to test, since they double the number of option combinations to try.

Source [<https://www.infoq.com/news/2016/02/compact-strings-Java-JDK9>]

# A Bit of History ...

**OptimizeStringConcat** - introduced by Java 6 u 18

➤ fragile feature leading the code to fall outside the **AST** sometimes

```
private int counter;
```

```
@Benchmark
public String inlineSideEffect() {
    return new StringBuilder().append(counter++).toString();
}
```

JDK 8u5

2,2 ns/op

```
@Benchmark
public String spliceSideEffect() {
    int cnt = counter++;
    return new StringBuilder().append(cnt).toString();
}
```

1,1 ns/op

Source [<https://bugs.openjdk.java.net/browse/JDK-8043677>]